File Encoding Step by Step — A Tutorial

This is the third, most detailed version of the guide on encoding a file into a DataShield stream — "for technical-college students": every term is introduced from scratch, every step is walked through with numbers, tables, and a complete worked example. The formal monograph is `EncoderGuide.Academic.en.md`; the short engineering retelling is `EncoderGuide.Engineer.en.md` (their section numbering matches each other; this tutorial uses its own, lesson-style numbering). A general overview of the format is in `AlgorithmGuide.en.md` (sections 2–4); the reverse process is the `DecoderGuide.*` family; assembling the file on the receiving side is the `AssemblyGuide.*` family.

After reading this tutorial you should be able to explain in your own words: what the encoder's output actually is, where every number comes from (`N`, `M`, `T`, the header count), how packets and their hashes are built, and why ECC volumes exist and how they are spread across the stream.

1. Vocabulary: the words we cannot do without

Term	What it actually is
Encoder	The `FileEncoder` class: turns a file into a list of 75-byte packets.
Packet	Exactly 75 bytes: the indivisible unit of the stream. Two kinds — header and sector.
Header	The file's "cap" packet: 51 bytes of fields + 24 bytes of the `H5` hash.
Sector	A packet carrying data: 2 bytes of number + 64 bytes of payload + 9 bytes of the `D3` hash.
Payload	64 bytes of content: a chunk of the file (data volume) or redundancy (ECC volume).
`N`	Number of data volumes: `N = max(1, ceil(FileSize / 64))`.
`M`	Number of ECC volumes: `M = max(1, ceil(N · eccPercent / 100))` when `eccPercent ≥ 1`, else 0.
`T`	`T = N + M` — the total number of data sectors in the stream.
`H1`–`H4`	Header fields: name (14 bytes), size (3), SHA-256 of the file (32), ECC volume count (2).
`H5`	Header hash: `Trunc24(SHA-256(H1-H4))` — 24 bytes, the "seed" for sector hashes.
`H3`	SHA-256 of the original file's contents — the future judge of assembly on the decoder side.
`D1`–`D3`	Sector fields: number (2 bytes), payload (64), hash (9).
ECC	Redundancy: a Reed–Solomon erasure code over the field GF(2^{16}).
ECC percent (`eccPercent`)	Setting: how much redundancy to add. Default 10%.
Header percent (`headerPercent`)	Setting: how often to repeat the header in the stream. Default 3%, minimum 3 copies.
FEC stream	The resulting packet stream: Base64 text (`.DataShield.txt`) or raw bytes (`.DataShield.bin`).

2. What the encoder does: the big picture

Input: the file's contents (bytes) and its name. Output: a list of 75-byte packets, later turned into text or a binary file. The pipeline, step by step:

```
file
   ├─ 1. checks: size ≤ 16,777,215 bytes, percents ≥ 0
   ├─ 2. file name → 14-byte H1 field
   ├─ 3. SHA-256 of contents → H3
   ├─ 4. dimensions: N, M, T = N + M (check T ≤ 65,535)
   ├─ 5. slicing: N payloads of 64 bytes (the last one zero-padded)
   ├─ 6. ECC: M redundant payloads (Reed-Solomon)
   ├─ 7. header: H1-H4 → 51 bytes, hash H5 → the header packet
   ├─ 8. sectors: T packets of the form D1+D2+D3
   ├─ 9. arrangement: headers at the start, the end, and evenly between sectors
   └─ 10. output: List<byte[]> → Base64 text or a binary file (PacketIO)
```

Two percents are configured in the constructor: `new FileEncoder(eccPercent: 10, headerPercent: 3)`.

3. Format limits — three hard ceilings

Limit	Value	Reason
File size	≤ 16,777,215 bytes (~16 MiB)	the `H2` field is 3 bytes
Total volumes `T = N + M`	≤ 65,535	the GF(2^{16}) field size of the Reed–Solomon code
Packet size	exactly 75 bytes	the format: 2 + 64 + 9 (sector) or 51 + 24 (header)

Exceeding any of them throws an `InvalidOperationException` with a clear message; the encoder never emits a "partially correct" stream.

4. Step one: the file name in 14 bytes (the `H1` field)

The name field in the header is just 14 ASCII characters. Packing rules (`FileNameCodec.Pack`):

1. Only the file name without the directory is taken (`Path.GetFileName`).
2. The name is split at the **first** dot: left — base, right — extension. The extension is preserved whole, even a compound one: `.tar.gz` stays `.tar.gz`.
3. Only the base is quantized (truncated). On truncation a `~` marker is appended to the base.
4. If less than 2 bytes remain for the base (1 character + marker) — the name is unrepresentable, exception.

Examples:

Original name	Packed H1	What happened
`report.pdf`	`report.pdf`	fits entirely (10 characters)
`verylongname.txt`	`verylongnam~.txt`	base truncated to 10 + marker
`a.tar.gz`	`a.tar.gz`	compound extension preserved
`my.documents.2024.xlsx`	`my.documents~.2024.xlsx`	split at the first dot; everything after it is the extension

Edge cases: no dot, or the dot is the first character — the whole name is treated as the base.

5. Step two: SHA-256 of the contents (`H3`)

An ordinary SHA-256 — 32 bytes — is computed over the file's contents. This is the most important cargo the encoder puts into the header: it is by this hash that the decoder will later prove it assembled the file bit for bit. Until then, `H3` is just a number; its role is explained in `AssemblyGuide.Tutorial.en.md`.

6. Step three: the dimensions `N`, `M`, `T`

Three formulas you should be able to compute by hand:

```
N = max(1, ceil(FileSize / 64))          — data volumes
  M = max(1, ceil(N · eccPercent / 100))   — ECC volumes (0 if eccPercent = 0)
  T = N + M                                — sectors in total
```

A table for `eccPercent = 10`:

FileSize, bytes	N	M	T
1	1	1	2
64	1	1	2
65	2	1	3
1000	16	2	18
100,000	1563	157	1720

Note the small files: even a 1-byte file needs one data volume, and at 10% ECC a minimum of one ECC volume — a redundancy of 6400%. This is deliberate: the `max(1, …)` formula guarantees that ECC mode really recovers losses even on tiny files.

7. Step four: slicing into payloads

The file is cut sequentially into 64-byte chunks; there are `N` of them. The last chunk is special: if the file size is not a multiple of 64, it holds fewer real bytes, and the tail is **zero-padded** to a full 64. Example: a 1000-byte file → 15 full chunks + a 16th chunk with 40 bytes of data and 24 zeros. The decoder will later cut these zeros off — the file size is known from the header (`H2`).

8. Step five: ECC volumes — Reed–Solomon redundancy

If `M > 0`, the encoder computes `M` redundant payloads by the rules of a Reed–Solomon erasure code over GF(2^{16}). In plain terms:

- One field symbol is 2 bytes. A 64-byte payload is **32 symbols**, and all 32 positions are processed independently, column by column.
- For each position a "column" is assembled: one symbol from every data volume. The code appends `M` ECC symbols to the end of the column — linear combinations of the data symbols over GF(2^{16}).
- The computed ECC symbols are laid back into their ECC volumes. Result: `M` new 64-byte payloads.

Why this matters: if any `k ≤ M` data volumes are lost in transit, the decoder reconstructs them from the surviving data and ECC volumes. The recovery condition is simple: **the number of lost data volumes ≤ the number of available ECC volumes**. Lost ECC volumes do not hurt at all — recovery simply does not use them.
The field's boundary condition: `N + M ≤ 65,535`. The GF(2^{16}) math itself lives in the reference module `RsRaid16` and is never modified.

9. Step six: the header packet and the `H5` hash

The header is assembled from the fields:

Field	Size	Offset	Content
`H1` name	14	0	the packed name (section 4)
`H2` size	3	14	FileSize, little-endian
`H3` SHA-256	32	17	hash of the contents
`H4` ECC volumes	2	49	M, little-endian

51 bytes in total. Then `H5 = Trunc24(SHA-256(these 51 bytes))` is computed — a full SHA-256 of which the **first 24 bytes** (192 bits) are kept. The header packet = 51 bytes of fields + 24 bytes of `H5` = the same 75 bytes as a sector.
Why `H5`: it is simultaneously an integrity check for the header (the decoder verifies it autonomously, with no keys or external data) and a "seed" — the key with which all sectors of this file are signed (section 10).

10. Step seven: data sectors (`D1 + D2 + D3`)

Each of the `T` payloads (N data + M ECC) becomes a sector:

Field	Size	Offset	Content
`D1` number	2	0	0 … T−1, little-endian
`D2` payload	64	2	64 bytes of content
`D3` hash	9	66	`Trunc9(SHA-256(H5 ‖ D1 ‖ D2))`

`D3` is a SHA-256 truncated to 9 bytes (72 bits) over the concatenation: 24 bytes of `H5` + 66 bytes of sector content. The key trick: **without `H5` this hash cannot be verified**. A sector is riveted to its own file: a foreign packet, even with a correct number and plausible content, will not pass the check. Result: `T` sectors of 75 bytes each.

11. Step eight: how many headers and where they go

The decoder needs the header to make any sense of the sectors (it needs `H5`). So the header is repeated:

```
copy count = max(3, ceil(T · headerPercent / 100))
```

Always a minimum of 3: the beginning, the middle, and the end of the stream. The arrangement (`ArrangePackets`):

1. The first packet of the stream is a header.
2. The last packet of the stream is a header.
3. The intermediate copies are spread evenly: interval `= max(1, T / (intermediates + 1))` ; a header is inserted after every `interval` -th sector while copies remain.

Examples:

T	headerPercent	Copies	Interval	Stream (H = header)
2	3	3	—	`H D0 D1 H`
18	3	3	9	`H D0…D8 H D9…D17 H`
200	3	6	40	`H D0…D39 H D40…D79 H … H D160…D199 H`

Why this matters: even if a chunk is cut out of the middle of the stream, at least one header copy almost certainly survives, and the decoder can identify the file.

12. Step nine: the output format

The encoder returns a `List<byte[]>` — ordered 75-byte packets. Then two formats are available (`OutputFormat`):

Format	Extension	Inside
Base64	`.DataShield.txt`	text: one line per packet, exactly 100 Base64 characters per line; optionally decorative framing lines `>[name][size][SHA-256:hex]` at the start and `<[…]` at the end
Binary	`.DataShield.bin`	raw packets back to back, no separators: 75 × packet-count bytes

Why a Base64 line is exactly 100 characters: 75 bytes = 25 triplets, each triplet encodes into 4 characters; 75 divides by 3 exactly, so `=` padding characters never occur. The decorative lines are for humans only: the decoder ignores them (the filter discards non-Base64 characters).
Writing to a file goes through `PacketIO.WriteFile`; the same class reads back (`ScanFile`), detecting the format by extension.

13. Progress and cancellation

`Encode` accepts an `IProgress<CodecProgress>` and a `CancellationToken`. The global 0…100 scale is divided into phases:

Phase	Range	What happens
Data preparation	0–10	checks, SHA-256, slicing
ECC encoding	10–75	Reed–Solomon (the heaviest part)
Packet building	75–100	assembling the `T` sectors
Done	100	`Done`

Cancellation is checked between phases and inside the ECC loop; on cancel an `OperationCanceledException` flies — a half-finished stream is never returned.

14. Encoding statistics (`EncodeStats`)

`EncodeWithStats` returns, along with the packets, a summary: file size, SHA-256, `N`, `M`, total packet count, and how many of them are header copies. Handy for UI and logs: you can show the user "18 sectors + 3 headers = 21 packets".

15. Memory hygiene

After assembling the packets, the encoder **zeroes out** the intermediate buffers: all N data payloads, all M ECC payloads, and the header bytes. The packets themselves, of course, live on — they are the result. The point of the wipe: no extra copies of the file's contents linger in memory longer than necessary.

16. Stream input

Besides a byte array, the encoder accepts a `Stream` — and reads it whole (SHA-256 and RS require the full contents). Seekable streams are read directly with a remaining-length check; non-seekable ones are copied through an intermediate buffer. The "≤ 16 MiB" check works in both cases. The stream is not closed by the encoder.

17. A complete worked example: a 1000-byte file, 10% ECC, 3% headers

Computing everything step by step:

1. Size 1000 ≤ 16,777,215 — OK.
2. `N = ceil(1000/64) = 16` (15 × 64 = 960; a 16th chunk is needed).
3. `M = max(1, ceil(16 · 10 / 100)) = max(1, 2) = 2`.
4. `T = 18 ≤ 65,535` — OK.
5. Payloads: 15 full 64-byte chunks + chunk #15 with 40 bytes of data and 24 zeros.
6. ECC: 2 redundant payloads by the rules of section 8.
7. Header: `H1` = the packed name, `H2` = 1000, `H3` = SHA-256 of the file, `H4` = 2; `H5` = Trunc24(SHA-256(H1–H4)).
8. Sectors: #0–15 — data, #16–17 — ECC, each with its own `D3`.
9. Headers: `max(3, ceil(18·3/100)) = max(3, 1) = 3`; interval `= 18/(1+1) = 9` → insertion after sector #8.
10. The resulting stream: `H D0…D8 H D9…D17 H` — **20 packets**.

Output sizes: the binary stream is 20 × 75 = **1500 bytes** (150% of the original); the Base64 text is 20 lines × 100 characters = **2000 characters** plus newlines. On large files the expansion tends to ≈132% in binary form (75/64 × 1.10 × 1.03) and ≈176% in Base64.

18. Edge cases

Case	What happens
Empty file (0 bytes)	`N = max(1, 0) = 1`: one data volume of pure zeros; the file "exists" and assembles
`eccPercent = 0`	`M = 0`, the ECC phase is skipped; loss recovery is impossible, only repeats
File > 16 MiB	`InvalidOperationException` immediately
`N + M > 65,535`	`InvalidOperationException` with a hint to reduce the size or the percent
A very long extension (> 12 bytes)	no budget remains for the base → `InvalidOperationException` from name packing
Negative percents	`ArgumentOutOfRangeException` from the constructor

19. What is guaranteed

1. The stream is either assembled fully and correctly, or the encoder throws — a "partial" output never exists.
2. The `H3` in the header is the exact SHA-256 of the contents: the decoder gets an impeccable judge of authenticity.
3. Every sector is signed with its own file's `H5` seed: foreign packets will not "stick" to this file.
4. With `M ≥ k`, the loss of any `k` data volumes is recoverable (within the correctness of RS over GF(2^{16})).
5. A minimum of 3 header copies: start, end, and evenly the middle.

20. Self-check questions

1. Why is a Base64 packet exactly 100 characters with no `=`? (section 12)
2. From which fields and how is `T` derived? (sections 6, 8)
3. Why do small files get a gigantic redundancy percent, and why is that right? (section 6)
4. How does `H5` differ from `H3`, and why are both needed? (sections 5, 9)
5. Why can a sector not be verified without its file's header? (section 10)
6. How many header copies will a stream with `T = 333` have at 3%? (section 11: `max(3, ceil(9.99)) = 10`)
7. What happens to a 1-byte file at `eccPercent = 10`? (section 18)
8. Why does the encoder zero the intermediate buffers? (section 15)
9. What are the format's three hard ceilings and where do they come from? (section 3)
10. What does a 1000-byte file turn into on output? (section 17)